Mid Full Stack Assessment

1. The Scenario

You are building the MVP for a Real-Time Financial Monitor.

The system needs to accept transaction data via an API, process it, and instantly update a live dashboard used by support agents.

Your Goal: Deliver a working, high-quality MVP that demonstrates high level coding standards and architectural thinking.

2. Core Requirements (MVP)

A. Data Model

All transactions in the system must adhere to the following structure (JSON):

JSON



B. Backend (.NET 8/9)

- **Ingestion API:** Create an HTTP endpoint (POST) to receive transaction data based on the model above.
- **Real-Time Layer:**
  - Implement a **WebSocket** server (using standard Microsoft.AspNetCore.WebSockets **OR** a library like SignalR) to broadcast updates to connected clients.
  - *Constraint:* Ensure your implementation handles multiple concurrent connections thread-safely.
- **Data Storage:**
  - Store the latest transactions in **Memory (RAM)** or **SQLite**.
  - Keep the data consistent and handle concurrency (avoid race conditions during reads/writes).

C. Frontend (React + TypeScript)

The client application must demonstrate a clear separation of concerns by implementing **two distinct routes**:

1. **Route 1: Transaction Simulator (`/add`)**
   - A simple form or "Generator" button that creates mock transactions and sends them to your Backend Ingestion API via HTTP POST.
   - *Goal:* This simulates the external system feeding data into your engine.
2. **Route 2: Live Dashboard (`/monitor`)**
   - Connects to the Backend WebSocket.
   - Renders transactions in a grid/list as they arrive in real-time from the backend.
3. **Performance:**
   - The UI must remain responsive. If 100 transactions arrive quickly, the browser should not freeze.
4. **State & UX:**
   - Status indicators (e.g., distinct colors for "Failed" vs "Success").
   - Basic client-side filtering (e.g., "Show only Errors").

3. Unit Tests (TDD Approach)

Implement unit tests following a Test-Driven Development (TDD) approach. Cover transaction processing, concurrency handling, and storage logic. Tests must be clean, meaningful, and executable automatically.

4. Bonus: Cloud-Native & Distributed Architecture

You do not need to spin up a cloud cluster, but your code must be "Cloud Ready".

1. **Distributed Architecture:**
   - **The Challenge:** If we deploy this backend to 5 different pods (replicas), a client connected to Pod A won't see data sent to Pod B.
   - **The Task:**
     - **Architect:** Describe (in your README/ADR) how you would solve this synchronization problem
     - **Implement (Optional but recommended)**
2. **Deployment (DevOps):**
   - Add a `Dockerfile` optimized for production (small image size).
   - Add Kubernetes YAML manifests (`deployment.yaml`, `service.yaml`) showing how you would deploy this service.

5. Bonus – Enhanced UI Experience

Add animation when new transactions arrive and smooth transitions between status changes while maintaining performance and responsiveness.